GEEK-PS-01 | PRODUCT AND SYSTEM DESIGN

KIZUNA SIGNAL

Complete Product Architecture

An explainable pre-release intelligence engine for Japanese media entering India's regional, multilingual content markets.

WHETHER	WHERE	WHEN	HOW
Market potential	States and languages	Launch window	Platform and localization

Architecture baseline | Hackathon MVP to production evolution

24 October 2026

01 Executive Product Definition

Kizuna Signal converts public demand signals, content characteristics and Indian regional context into an explainable market-potential forecast and launch playbook.

Buyer decision: Which Japanese title, state, language, platform and launch window deserves investment?

Primary user

Japanese studios, IP owners, distributors and market-entry teams evaluating India before release.

Decision output

National and regional scores, confidence, comparable titles and a localized launch recommendation.

Evidence model

Public demand proxies first; partner outcomes later for commercial calibration.

Core differentiator

India is treated as multiple language and cultural markets rather than one national average.

Credibility rule

Report a calibrated market-potential score, not an unsupported promise of exact revenue.

Human control

Analysts can inspect, edit assumptions, disable sources and compare alternative scenarios.

Product loop

02 Complete Feature Architecture

The product is organized around a traceable decision journey: intake, evidence, intelligence, explanation, simulation and action.

Title intake

Projects; multilingual titles; synopsis, poster and trailer upload; content format; studio, cast and franchise metadata; release assumptions; completeness checks.

Signal collection

Google Trends, YouTube, Wikimedia, AniList, Census, DataMeet, CSV and partner-data adapters with caching, provenance and source health.

Content DNA

Synopsis embeddings; genres, themes and archetypes; audience fit; visual signals; cultural affinity and sensitivity flags.

Comparable engine

Semantic retrieval; business re-ranking; manual inclusions; historical evidence; optimistic, expected and conservative comp sets.

Market scoring

National, state, language and segment scores across content fit, momentum, engagement, regional opportunity and comparable performance.

Regional intelligence

India heat map; state ranking; language affinity; audience context; region comparison; state-specific explanations.

Audience intelligence

Likely audience groups; fandom intensity; mainstream crossover; family, youth and premium-market suitability.

Explainability

Positive and negative drivers; SHAP contributions; evidence links; missing-data impact; executive and analyst explanations.

Confidence controls

Intervals; source agreement; freshness; sample strength; ambiguity and out-of-distribution warnings; source disable and recalculation.

Scenario lab

Dubbing versus subtitles; platform, region, release window and budget alternatives; score and confidence deltas.

Launch playbook

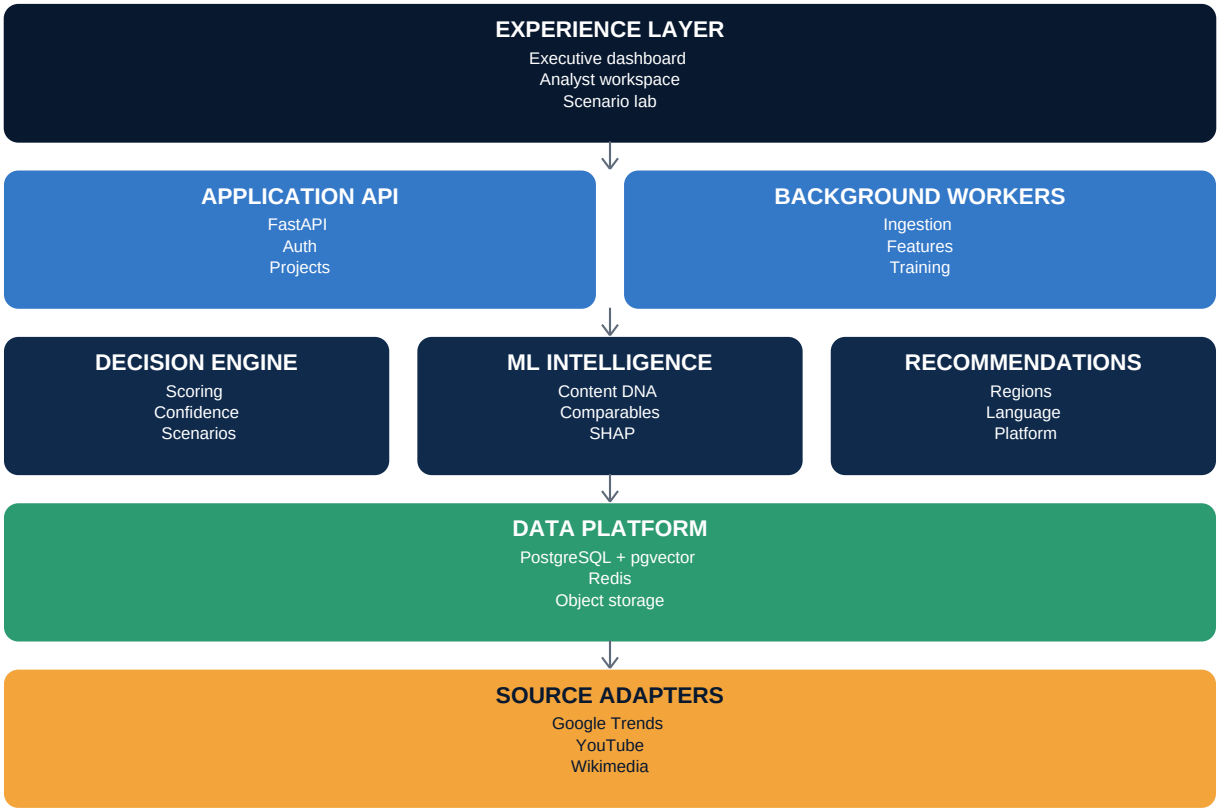
Priority states; languages; platform; campaign intensity; marketing themes; release timing; risks and mitigations.

Reporting

Executive brief; analyst PDF; CSV; chart export; saved versions; assumptions; citations and read-only sharing.

03 System Architecture

A modular monolith with background workers is the most stable hackathon foundation. Logical boundaries remain explicit so high-load modules can later become services without rewriting the product.



Architecture principles

- Keep the UI thin: all scoring and business logic live behind versioned APIs.
- Bind every prediction to a feature snapshot, model version and scoring configuration.
- Isolate every external data provider behind a source adapter.
- Continue with partial evidence when an optional source fails, but reduce confidence.
- Store immutable raw responses before normalization and feature engineering.
- Separate market potential from confidence; never hide uncertainty inside one score.

04 Application and API Layer

FastAPI forms the product boundary. Long-running work returns a job identifier and executes asynchronously, keeping the interface responsive and failures recoverable.

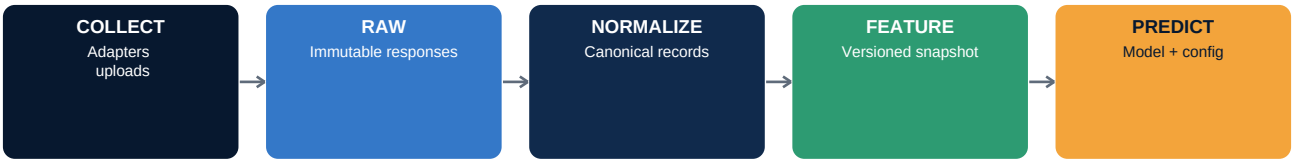
Domain	Responsibilities	Representative routes
Projects and titles	Create projects, titles, assets and assumptions; validate input completeness.	POST /projects; POST /titles; GET /titles/{id}
Source management	Start collection, inspect provenance, disable noisy sources and view health.	POST /sources/collect; GET /sources/health
Analysis	Run features, comparables, forecasts, confidence and explanations.	POST /titles/{id}/analyse; GET /analyses/{id}
Regions	Return state and language scores, evidence and comparisons.	GET /analyses/{id}/regions
Scenarios	Apply controlled overrides and calculate score, confidence and cost deltas.	POST /analyses/{id}/scenarios
Recommendations	Produce localization, platform, timing and campaign decisions with evidence.	GET /analyses/{id}/recommendation
Reports and jobs	Generate exports and expose asynchronous status and errors.	POST /reports; GET /jobs/{id}

Frontend contract

- Streamlit calls the same REST endpoints that a future React or Next.js frontend will use.
- The browser never receives source credentials, database credentials or model files.
- API responses include result version, confidence, provenance summary and warning codes.
- Idempotency keys protect create-analysis and report-generation requests from duplicates.

05 Data Platform and Lifecycle

Data moves through immutable raw storage, normalized evidence and reproducible feature snapshots. This prevents silent changes and allows every result to be audited.



Raw layer

Original API responses, exports and assets. Append-only, checksum-protected and retained for auditability.

Normalized layer

Canonical measurements with title, source, metric, geography, language, event time, collection time and provenance.

Feature layer

Model-ready values with feature definition version, observation cutoff, leakage status and source references.

Transactional store

PostgreSQL holds users, projects, observations, snapshots, predictions, scenarios, recommendations and audit events.

Vector retrieval

pgvector stores synopsis and content embeddings for exact or approximate comparable-title search.

Object storage

S3 or MinIO stores posters, reports, raw files and model artifacts using signed access URLs.

Minimum core entities

organizations, users, projects, titles, title_assets, source_queries, raw_observations, normalized_observations, feature_snapshots, comparables, model_versions, predictions, regional_predictions, prediction_drivers, scenarios, recommendations, reports, jobs and audit_events

06 External Source Architecture

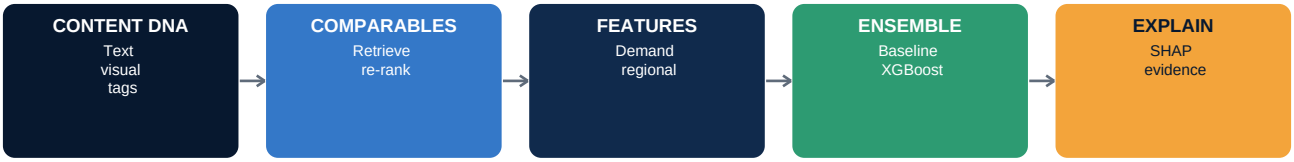
Each provider is independently replaceable. A shared adapter contract validates configuration, resolves title identities, collects evidence, normalizes records and returns quality and provenance metadata.

Source	Contribution	Stability guardrail
Google Trends	Relative search demand, momentum, seasonality and state interest.	Cache exports; track query variants; label 0-100 normalization; lower confidence for sparse terms.
YouTube	Trailer views, likes, comments, metadata and engagement velocity.	Respect quotas; separate global metrics from India-specific inference; store observation timestamps.
Wikimedia	Page-view growth and sustained information-seeking interest.	Treat as curiosity, not consumption; cache historical responses.
AniList	Synopsis, genres, tags, studios, relations, popularity and favourites.	Use popularity only as historical outcome; block post-release leakage for a predicted title.
Census India	Language and population context for addressable-market weighting.	Label table year and avoid treating demographic context as current behaviour.
DataMeet	State and district boundaries for regional visualization.	Version boundary files and validate state-name mapping.
Partner data	Licensing, spend, completion, conversion, revenue and regional outcomes.	Contracted schema, restricted access, encryption, retention and explicit usage purpose.

If a source fails, continue with available evidence, expose the missing source and reduce confidence. Never fabricate replacement data.

07 AI and Decision Pipeline

The intelligence layer combines interpretable baselines, tabular machine learning, semantic retrieval and explicit confidence modeling. A language model may explain verified results but must not invent scores.



Model components

- Content-fit model: synopsis, genres, themes, format and franchise signals.
- Demand model: search and attention levels, velocity, stability and seasonality.
- Regional model: state demand, language affinity and addressable-market context.
- Comparable model: performance patterns from semantically and commercially similar titles.
- Confidence model: completeness, agreement, freshness, similarity and model uncertainty.

Recommended techniques

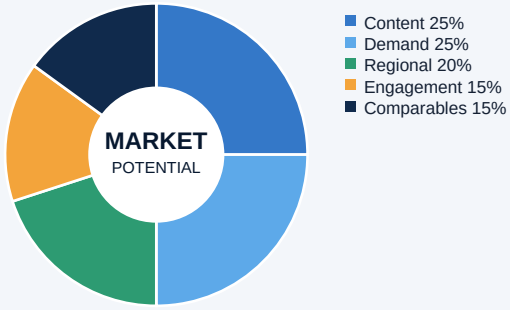
- Ridge or logistic regression as a transparent benchmark.
- XGBoost as the primary tabular learner.
- Sentence Transformers for synopsis embeddings.
- pgvector for comparable retrieval.
- Bootstrap, conformal or quantile intervals for uncertainty.
- SHAP for local and global feature explanations.

Comparable retrieval

Stage 1 retrieves a broad semantic candidate set. Stage 2 re-ranks by genre, audience, format, franchise, lifecycle and Indian evidence. Analysts can include or exclude a title, but each change is recorded.

08 Scoring, Confidence and Evaluation

Potential and confidence are separate outputs. High potential with weak evidence must look different from high potential supported by strong, consistent evidence.



Illustrative scoring configuration

Market Potential = 25% Content Fit + 25% Demand Momentum + 20% Regional Opportunity + 15% Engagement Quality + 15% Comparable Performance.

Weights are configuration, not code

Store every weighting scheme as a versioned scoring configuration so the system can reproduce prior outputs and test alternative policies.

Confidence factors

- Data completeness and freshness
- Cross-source agreement
- Comparable quality and count
- Query ambiguity and regional sample strength
- Model interval width and out-of-distribution distance

Evaluation framework

Metric	What it proves	Required split
Rank correlation	Whether higher-scored titles actually outperform lower-scored titles.	Time-based held-out titles
Top-three regional hit rate	Whether the system identifies the strongest regions.	Held-out title-region outcomes
Calibration error	Whether stated confidence matches observed reliability.	Unseen future-like samples
Baseline uplift	Whether the model beats simple popularity and genre baselines.	Same test set for all models
Decision time	Whether the product reduces analyst effort without reducing comprehension.	Pilot workflow measurement

09 Scenario and Recommendation Architecture

Scenarios apply controlled overrides to a frozen feature snapshot. They do not silently retrain the model or alter the base analysis.

Localization

Subtitles versus dubbing; Hindi-only versus multilingual; language-priority changes.

Distribution

OTT, theatrical, television, digital-first or hybrid release assumptions.

Geography

National launch, regional-first rollout or selected-state focus.

Timing

Alternative launch months, campaign lead times and seasonality assumptions.

Marketing

Budget bands, campaign intensity, fan-community and mainstream positioning.

Evidence

Disable a source, change comparable set or add partner information and inspect the confidence delta.

Recommendation contract

- Each recommendation contains the decision, expected benefit, evidence, confidence and principal risk.
- Business rules enforce safety and consistency; model outputs estimate opportunity and trade-offs.
- A language model may rewrite verified recommendation objects into readable prose, but cannot change numerical values.
- Human approval is required before a recommendation is marked accepted or exported as a launch decision.

10 Reliability and Failure Design

Stability is achieved by designing for incomplete evidence and predictable failure, not by assuming every provider and model will always work.

Failure	Expected behaviour	User-visible result
AniList unavailable	Use cached metadata or manual input; queue a retry.	Metadata-source warning; confidence penalty only if required fields remain missing.
Google Trends unavailable	Use approved export or latest cache; never fabricate current interest.	Freshness warning and reduced demand confidence.
YouTube quota reached	Use cached observation and defer refresh.	Engagement timestamp and partial-analysis status.
Weak regional evidence	Suppress unsupported state ranking or widen intervals.	National result remains available with explicit regional limitation.
Primary model fails	Run the validated baseline model and record fallback execution.	Baseline badge and wider uncertainty.
Report generation fails	Preserve analysis; retry asynchronously from a dead-letter queue.	Analysis remains usable; export status shows retrying.

External calls

Timeouts, bounded retries, exponential backoff, quota tracking and circuit breakers.

Background jobs

Idempotency, explicit states, partial completion, dead-letter handling and restart safety.

Database

Migrations, connection pooling, constraints, backups and tested restore procedures.

Release

Automated tests, feature flags, versioned APIs and rolling or blue-green deployment.

11 Security, Governance and Privacy

Unreleased scripts and trailers are commercially sensitive. Security is a product requirement, not a production-only enhancement.

Identity	Authorization	Content protection	Governance
- OIDC or OAuth 2.0 - Short-lived JWT access tokens - Optional multifactor authentication - Session revocation	- Organization and project isolation - Admin, manager, analyst and viewer roles - Least-privilege service accounts - Signed object access	- Encryption in transit and at rest - File validation and malware scan - No unreleased text in logs - Configurable retention and deletion	- Dataset and license registry - Model and scoring approval - Audit events for changes and downloads - Bias and performance checks by segment

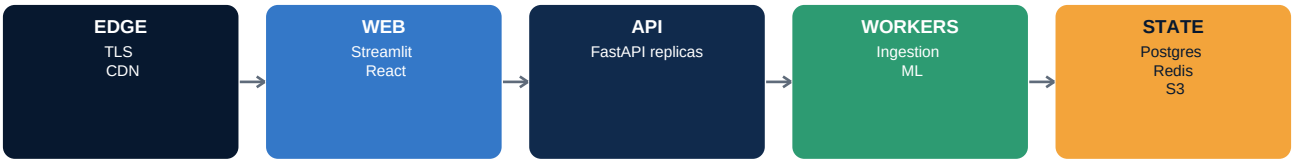
Evidence governance

- Every observation records its source, query, event time, collection time and transformation history.
- Every feature records its definition version and the source observations used to derive it.
- Every prediction records its feature snapshot, model version and scoring configuration.
- Every manual override records the user, time, previous value, new value and reason.
- Partner data is segregated by organization and never used for another partner without explicit authorization.

The product should support a decision; it should never obscure the evidence, uncertainty or human accountability behind that decision.

12 Deployment and Observability

The same logical design runs locally through Docker Compose and later moves to managed containers without changing the product contract.



Concern	Hackathon	Production evolution
Runtime	Docker Compose on one host.	Managed container service; Kubernetes only when operational scale justifies it.
Frontend	Streamlit and Plotly.	React or Next.js using the same versioned API.
State	PostgreSQL + pgvector, Redis and MinIO.	Managed PostgreSQL, Redis and S3-compatible object storage.
Workflows	Prefect worker or simple queue.	Horizontally scalable worker pools with scheduler and dead-letter queue.
Delivery	Manual or single CI deployment.	Automated tests, migrations, staged rollout and rollback.

Operational visibility

- OpenTelemetry traces and structured JSON logs
- Job duration, retry and partial-completion monitoring
- API latency, error and saturation metrics
- Feature drift, prediction drift and calibration tracking
- Source failure, quota and cache-hit monitoring
- Alerts tied to action thresholds rather than raw event volume

13 Technology Stack

The stack prioritizes Python productivity, interpretable modeling, open standards and a direct path from prototype to managed production.

Experience

Streamlit, Plotly; later React or Next.js

Application

Python, FastAPI, Pydantic, SQLAlchemy, Alembic

Data

Pandas or Polars, PostgreSQL, pgvector, Redis

Machine learning

scikit-learn, XGBoost, Sentence Transformers, SHAP

Workflow

Prefect; asynchronous workers and scheduled refreshes

Storage

MinIO locally; S3-compatible managed storage in production

Quality

Pytest, type checks, schema tests, leakage tests, Evidently

Operations

Docker, GitHub Actions, OpenTelemetry, Prometheus, Grafana, Sentry

Avoid during the hackathon

- Kubernetes and service meshes
- Many independently deployed microservices
- Real-time streaming infrastructure
- Training large foundation models
- Scraping platforms without clear permission
- A chatbot that hides evidence behind generated prose

14 Implementation Roadmap

A narrow, reliable vertical slice is more valuable than a wide set of disconnected screens. Build one analysis all the way from title input to exported decision.

Phase	Scope	Exit evidence
Hackathon core	Title input; cached public sources; Content DNA; five comparables; national and state scores; confidence; explanations; one scenario; launch playbook; PDF export.	A deterministic four-minute demo that runs without live-provider dependency.
Pilot hardening	Authentication; project isolation; partner dataset import; model back-testing; stronger data validation; audit logs; workflow monitoring.	10-20 historical decisions reproduced with agreed metrics and analyst review.
Operational product	Managed infrastructure; automated refresh; model registry; drift monitoring; approval workflow; report sharing; regional outcome feedback.	Stable multi-user operation with support, recovery and governance procedures.
Commercial intelligence	Licensed viewership, spend, completion and revenue outcomes; platform economics; portfolio optimization; enterprise integrations.	Forecasts calibrated to real partner outcomes and decision value.

Hackathon demonstration sequence

1	Select an unreleased or mock Japanese title.
2	Show metadata, Content DNA and source coverage.
3	Reveal national potential, confidence and top three regions.
4	Inspect comparable titles and positive and negative drivers.
5	Disable one noisy source and show confidence recalculation.
6	Compare subtitle and multilingual dubbed scenarios.
7	Generate and export the launch playbook.

15 Research and Reference Register

Primary documentation supports the challenge, data access and technology choices. Commercial systems are product benchmarks only; their proprietary models and data are not dependencies.

Type	Reference	URL
Official challenge	Thapar Japan Hackathon - GEEK-PS-01	https://hack.tslasconnect.com/problem-statements?ps=geek-ps-1
Challenge partner	Geek Pictures IP and global business	https://geekpictures.co.jp/en/ip/
Search evidence	Google Trends methodology	https://support.google.com/trends/answer/4365533
Search API	Google Trends API alpha	https://developers.google.com/search/apis/trends
Video evidence	YouTube Data API	https://developers.google.com/youtube/data/getting-started
Page views	Wikimedia Analytics API	https://doc.wikimedia.org/generated-data-platform/aqs/analytics-api/reference/page-views.html
Anime metadata	AniList API documentation	https://github.com/AniList/docs
India geography	DataMeet maps	https://github.com/datameet/maps
Demographics	Census of India data tables	https://censusindia.gov.in/census.website/data/census-tables
Research	Anime popularity prediction using multimodal deep learning	https://arxiv.org/abs/2406.16961
Open-source reference	Waku-Waku Anime Analytics	https://github.com/foniss/anime-analytics-dashboard
Open-source reference	SC1015 Anime Success Prediction	https://github.com/ztjhz/SC1015-Project
Commercial benchmark	Parrot Analytics concept testing	https://www.parrotanalytics.com/insights/why-most-content-investments-fail/
Commercial benchmark	Ormax Media content testing	https://www.ormaxmedia.com/insights/stories/content-testing-frequently-asked-questions.html
Commercial benchmark	Vault AI platform	https://vault-ai.com/platform
Commercial benchmark	Cinelytic forecasting	https://www.cinelytic.com/faq/

Commercial accuracy claims are vendor-reported and not directly comparable because each organization predicts different outcomes using different definitions and proprietary datasets.

16 Final Architecture Position

Kizuna Signal should begin as a disciplined decision-support product, not an overextended prediction platform.

Public evidence + Content DNA + Indian regional context -> explainable potential, confidence and launch action.

Build now

One reliable vertical slice: input, evidence, comparables, regional score, confidence, explanation, scenario and export.

Validate next

Use 10-20 anonymized historical title decisions to compare ranking quality, regional insight and analyst time saved.

Scale later

Add licensed outcomes, managed infrastructure, automated monitoring and enterprise integrations only after pilot evidence.

Architecture success criteria

- A prediction can be reproduced from stored evidence, feature snapshot, model and configuration.
- Post-release information cannot leak into a pre-release model evaluation.
- The product remains usable when an optional external source is unavailable.
- The hackathon prototype can evolve without discarding its API, data or model boundaries.
- Every recommendation exposes evidence, uncertainty and human accountability.
- Business value is measured through ranking quality, regional insight and decision time saved.

Recommended pilot ask

Provide 10-20 anonymized historical title decisions and outcomes for a four-week evaluation of ranking, regional insight, calibration and analyst time saved.